

SAATHI — Team 1

Complete Knowledge Engineering, Ingestion, Chunking and Versioning Work Package

Operating rules

- Keep the existing ingestion package, versioning package, current AI/backend, and the ~1.25 GB RAGA model. Do not delete or retrain existing assets before benchmarking them.
- Use the existing 600+ MB government-source dataset as the controlled starting corpus. Every source must retain provenance; do not silently add unverified web data.
- Work in separate Git branches and reproducible Colab/Kaggle notebooks. Every experiment must record dataset/model/index version and metrics.
- Never replace a working baseline without a measured comparison.

Mission

- Build the highest-quality regulatory knowledge layer: source registry → extraction/OCR → structure → clause-aware chunks → metadata → deduplication → versioning → quality gates.

Step 1 — Inventory the 600+ MB corpus

- Enumerate every file and source domain. Generate a manifest with source URL, title, document type, local path, hash, file size and collection date.
- Classify documents into Indian Standards, QCOs, schemes, certification material, regulations, manuals/FAQs and other sources.
- Mark authoritative vs supporting sources. Do not treat a general webpage as equivalent to an official standard.

Step 2 — Extraction and OCR

- Run the existing document_ingestion_v2 pipeline first; compare its output rather than rebuilding it.
- Measure text extraction success, scanned-PDF detection, OCR usage, page preservation, table extraction and clause detection.
- Flag password-protected, corrupted, zero-text and OCR-failed documents for manual review. Never pretend a failed extraction succeeded.

Step 3 — Canonical document schema

- Create one immutable document registry and one chunk schema used by all teams.
- Required chunk fields: chunk_id, document_id, standard_number, title, document_type, version, publication_date, effective_date, clause, section, page, text, source_url, superseded status.
- Do not guess missing metadata; use null/unknown and create a QA report.

Step 4 — Clause-aware chunking

- Prefer clause/section boundaries over arbitrary character splitting.
- Preserve enough context for definitions, scope, requirements, exceptions and tables.
- Create parent-child relationships where a clause is too large to fit one chunk.

Step 5 — Versioning and supersession

- Merge the existing versioning output with the canonical registry.
- Represent current, superseded, amendment and unresolved relationships explicitly.

- Prevent old vectors from masquerading as current requirements.

Step 6 — Knowledge-quality gates

- Detect duplicate documents/chunks, malformed text, broken URLs, missing pages, OCR errors and conflicting metadata.
- Produce a quality scorecard and a list of documents requiring manual review.
- Deliver a frozen corpus version to Team 2.

Deliverables

- corpus_v1.jsonl or database export
- document_registry_v1
- version_registry_v1
- source_manifest.csv
- knowledge_quality_report
- reproducible ingestion notebook/script
- README explaining exactly how the corpus was generated

Acceptance test

- Another person can rebuild the corpus from the manifest and obtain the same document/chunk IDs.
- Every retrieved chunk can be traced to source document, page and clause where available.
- No unsupported metadata is invented.